

의사결정나무와 앙상블



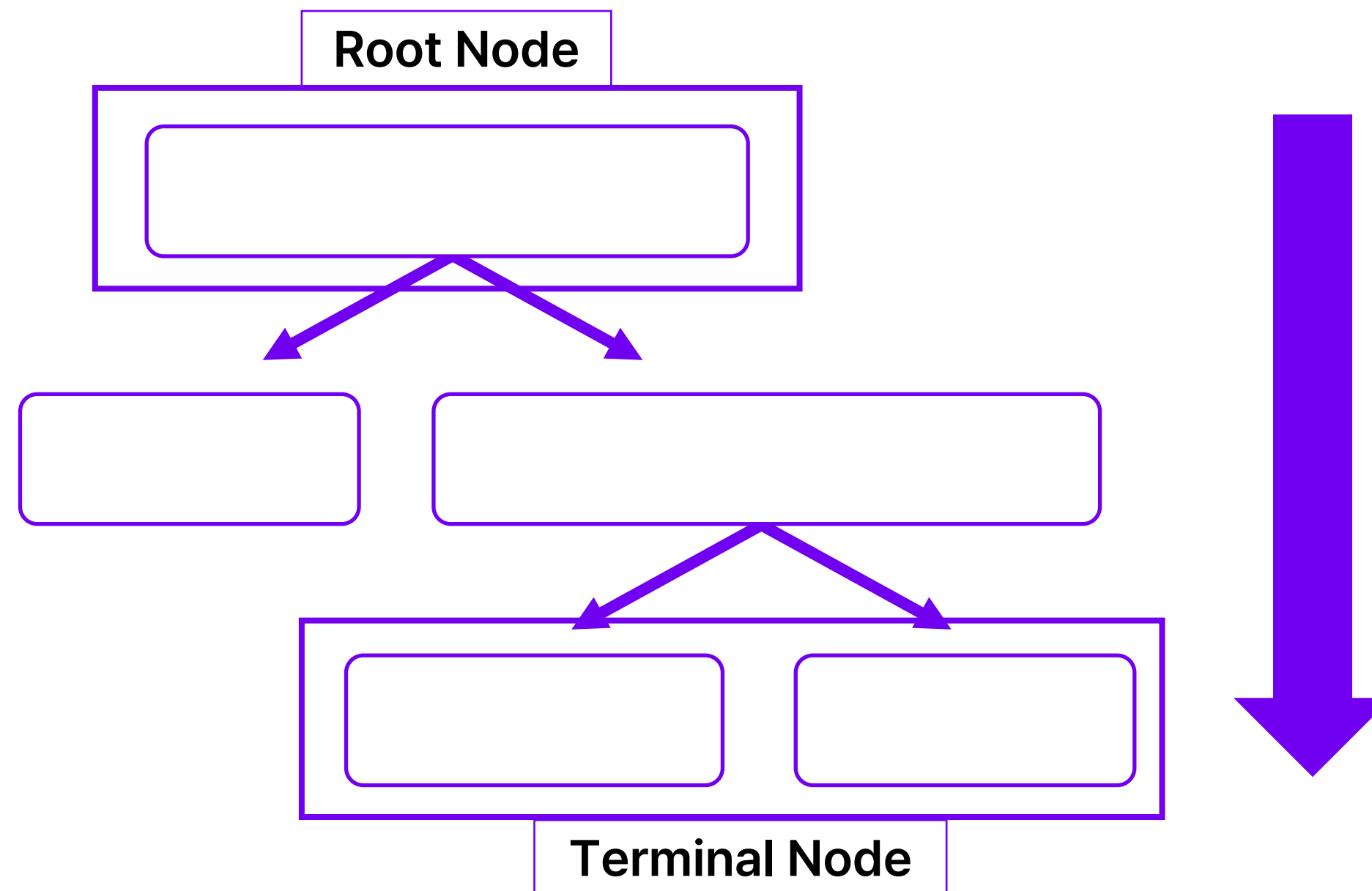
문제 제시

- 회귀 : 단순/다중 선형 회귀, 다항 회귀...
- 분류 : 로지스틱 회귀, 서포트 벡터 머신, 나이브 베이즈 분류기...
 - 복잡하고 이해하기 어려운 모델 원리들
- 해결 방안 : 시각적으로 쉽게 이해할 수 있는 의사결정나무(Decision Tree) 모델



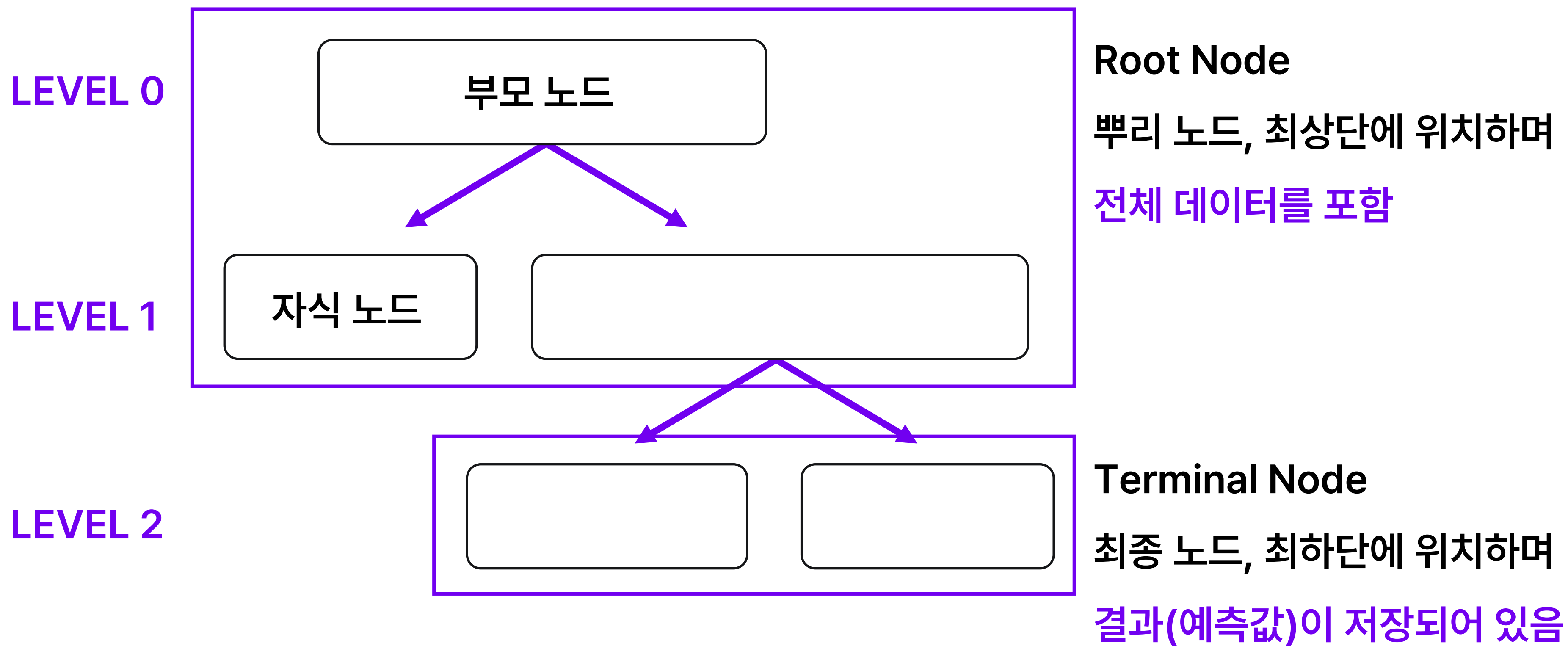
반복적인 분할을 통해 데이터를 단계적으로 분류 하는 모델 스무고개와 같이 **특정 질문들을 통해 정답을 찾아 감**

최상단의 Root Node에서 마지막 Terminal Node까지 **아래 방향으로 진행**





의사결정나무의 Node





의사결정나무의 Node

LEVEL 0

부모 노드

LEVEL 1

자식 노드

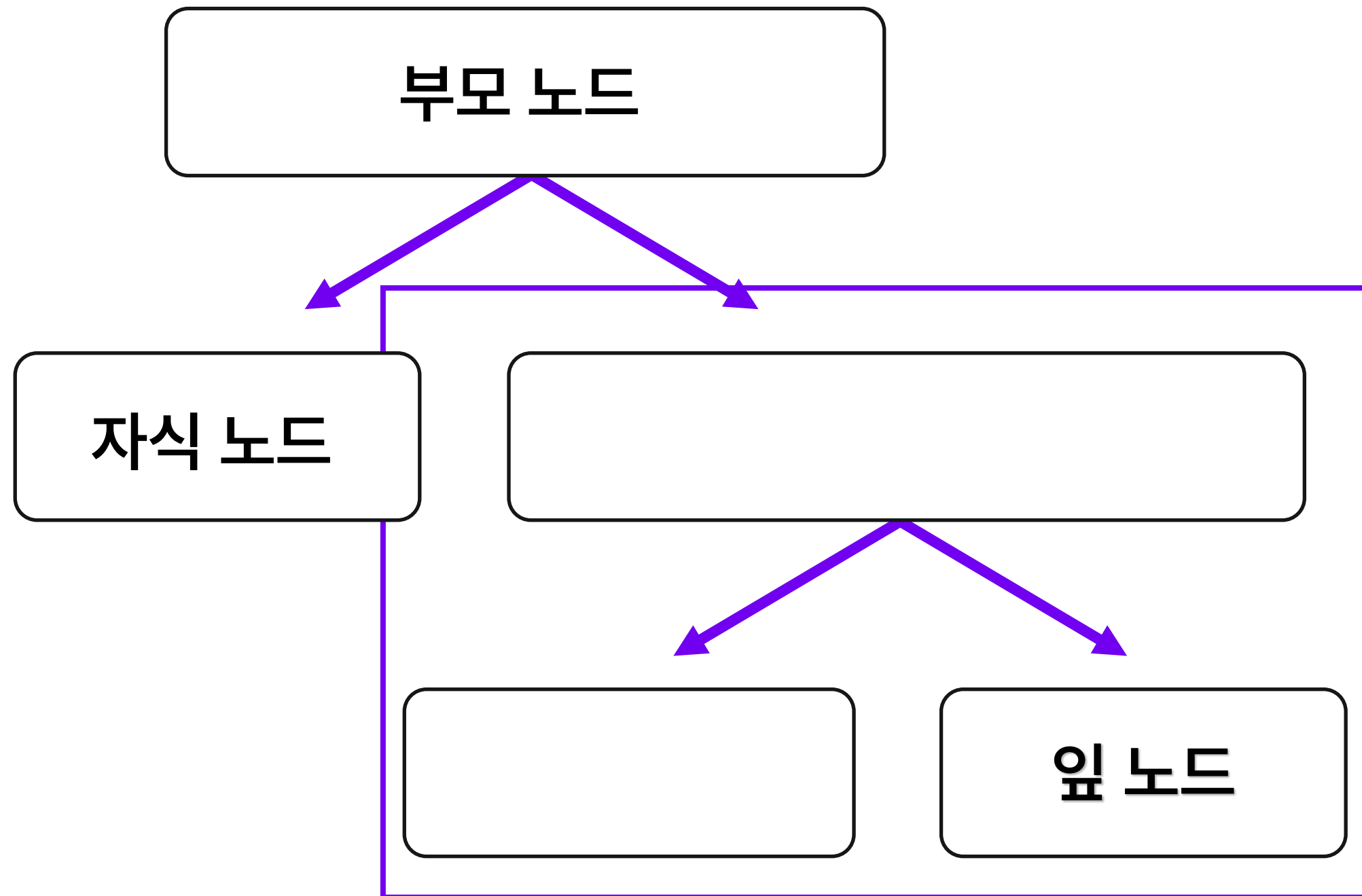
LEVEL 2

잎 노드

서브 트리(Sub-tree)

전체 중 일부 트리

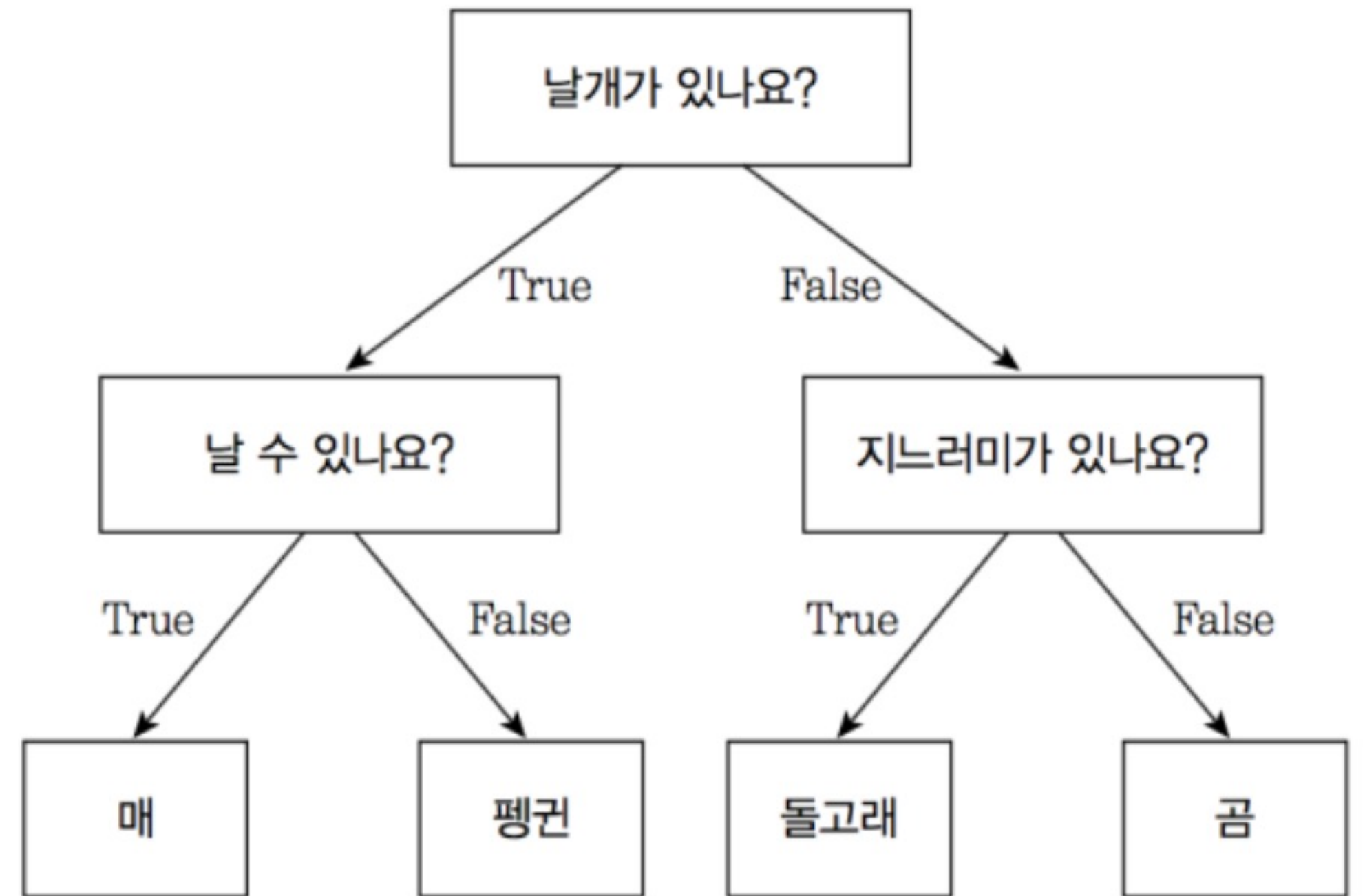
가지치기에 이용 됨





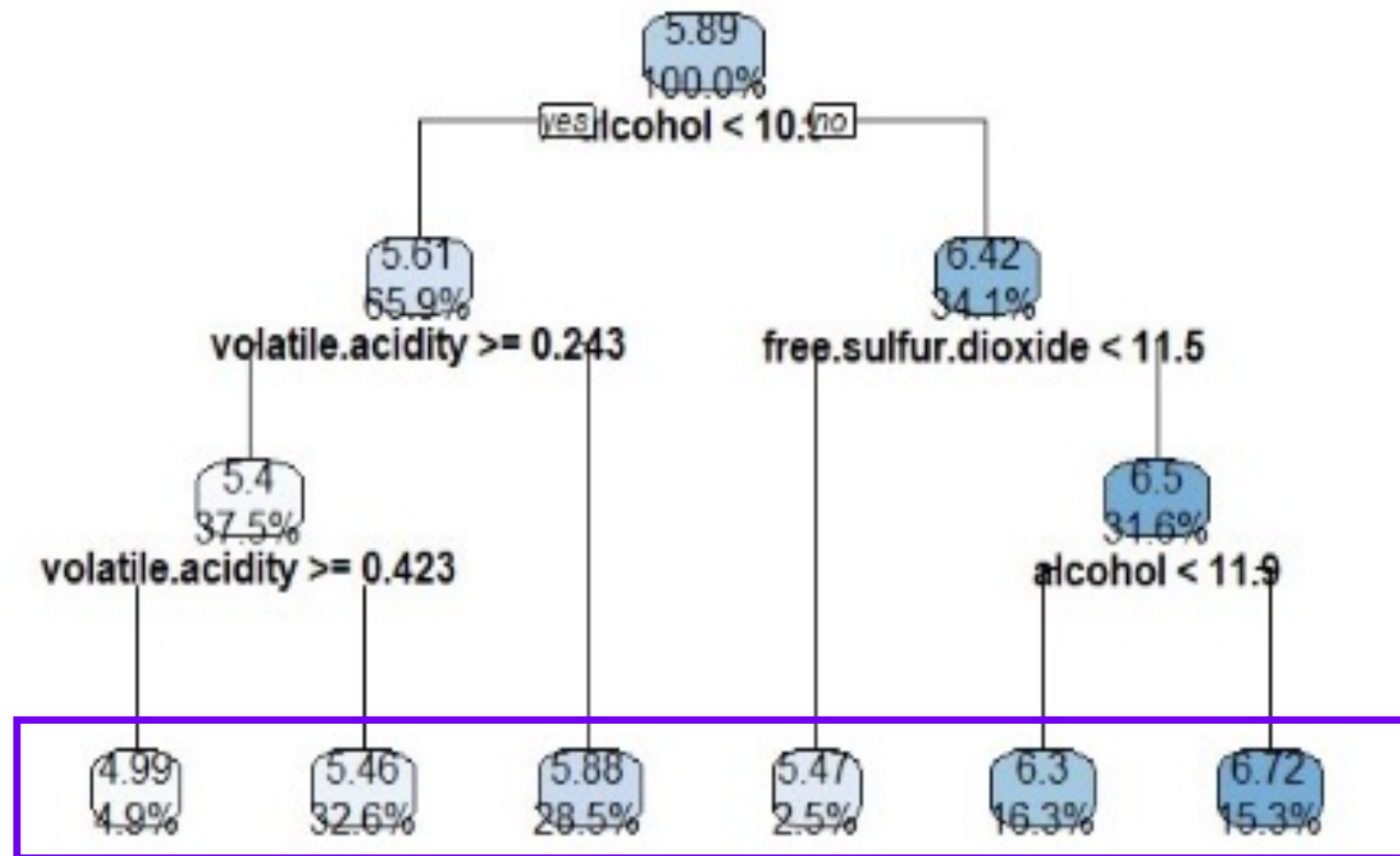
의사결정나무 특징

- 직관적이고 이해하기 쉬움
- 비선형 관계 모델링 가능
- 데이터 전처리에 덜 민감
- 나무가 깊어지며, 과적합이 발생할 수 있음

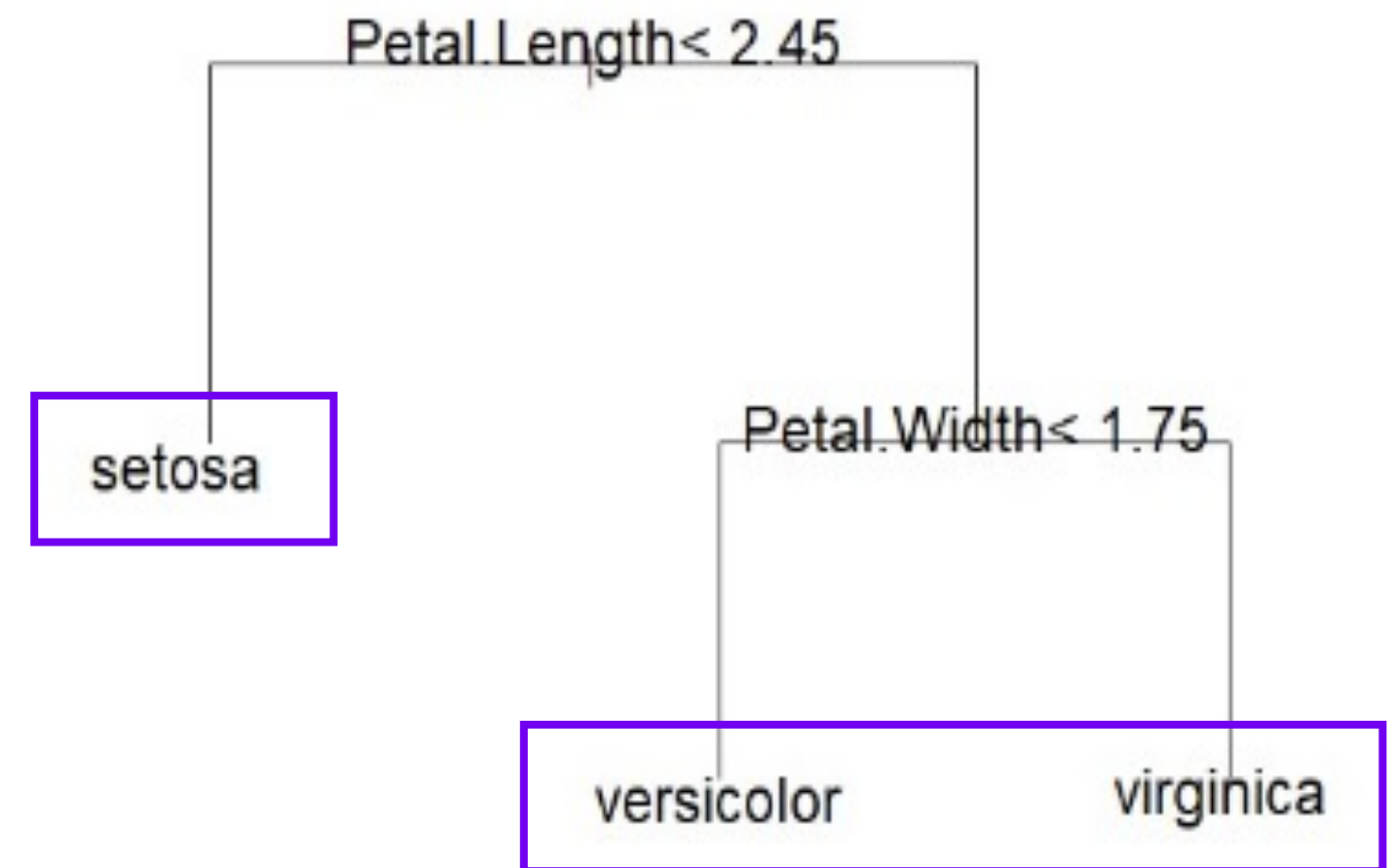




의사결정나무는 지도학습 알고리즘으로, 회귀와 분류 모두 적용 가능



회귀 의사결정나무



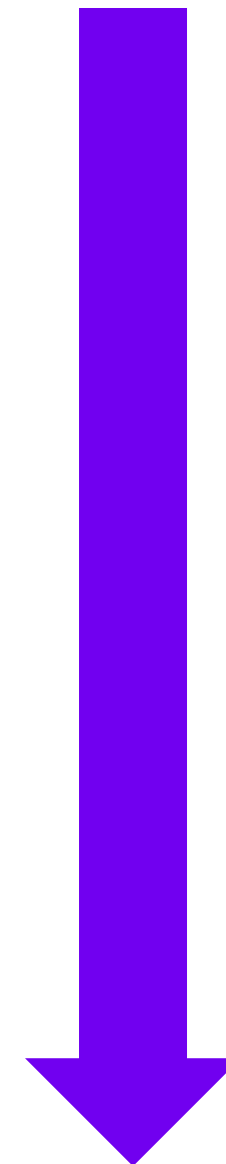
분류 의사결정나무



의사결정나무 - 회귀

- 회귀를 위한 의사결정나무 모델은 어떻게 학습을 진행하고, 결과값을 예측할까?

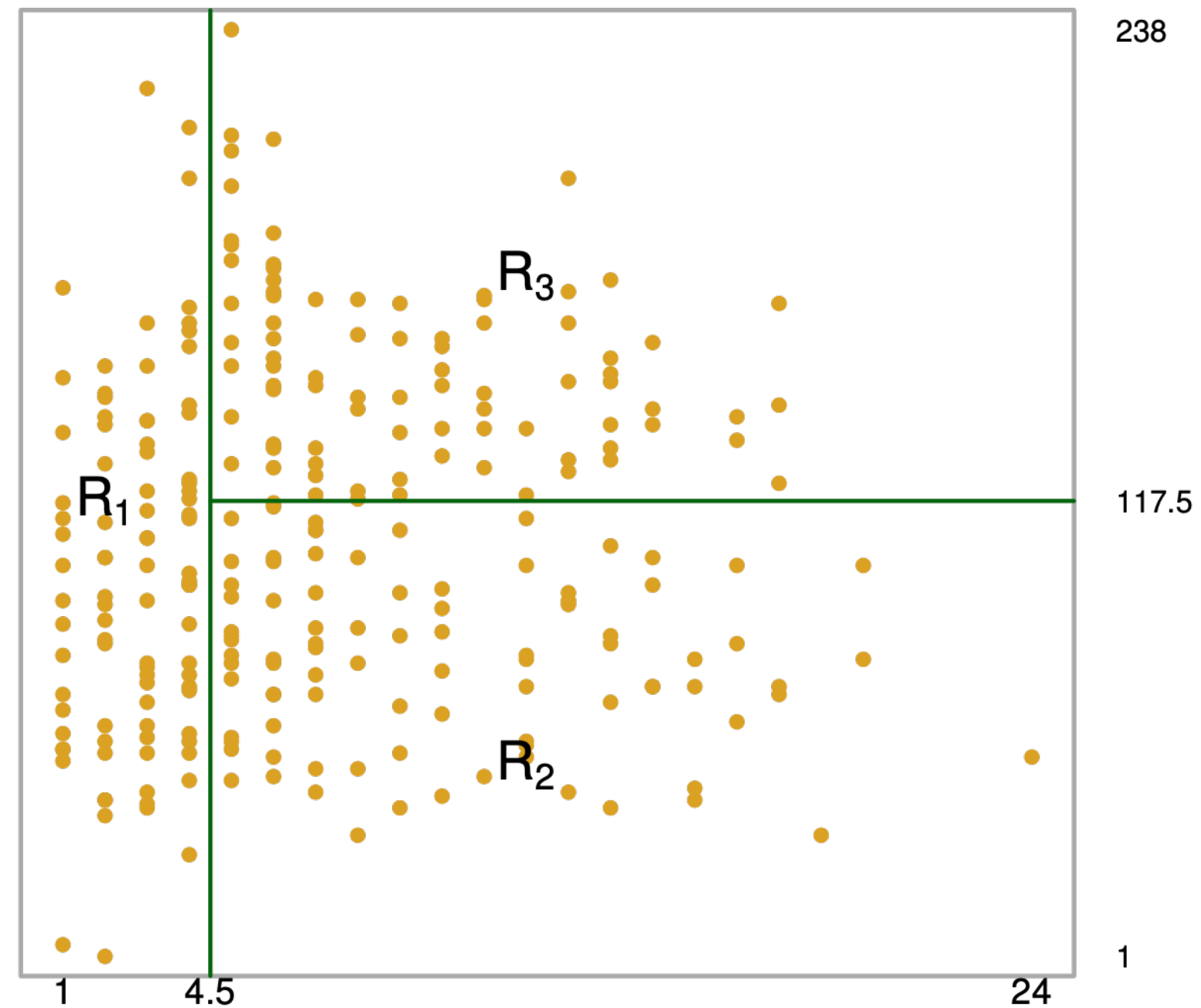
→ 위에서 아래로 진행될수록 데이터의 구역을 나누자!





의사결정나무 - 회귀

- 겹치지 않는 구역으로 데이터를 나눔
→ 어떤 구역으로 어떻게 나누어야 할까?





어떤 구역으로 어떻게 나누어야 하는가?

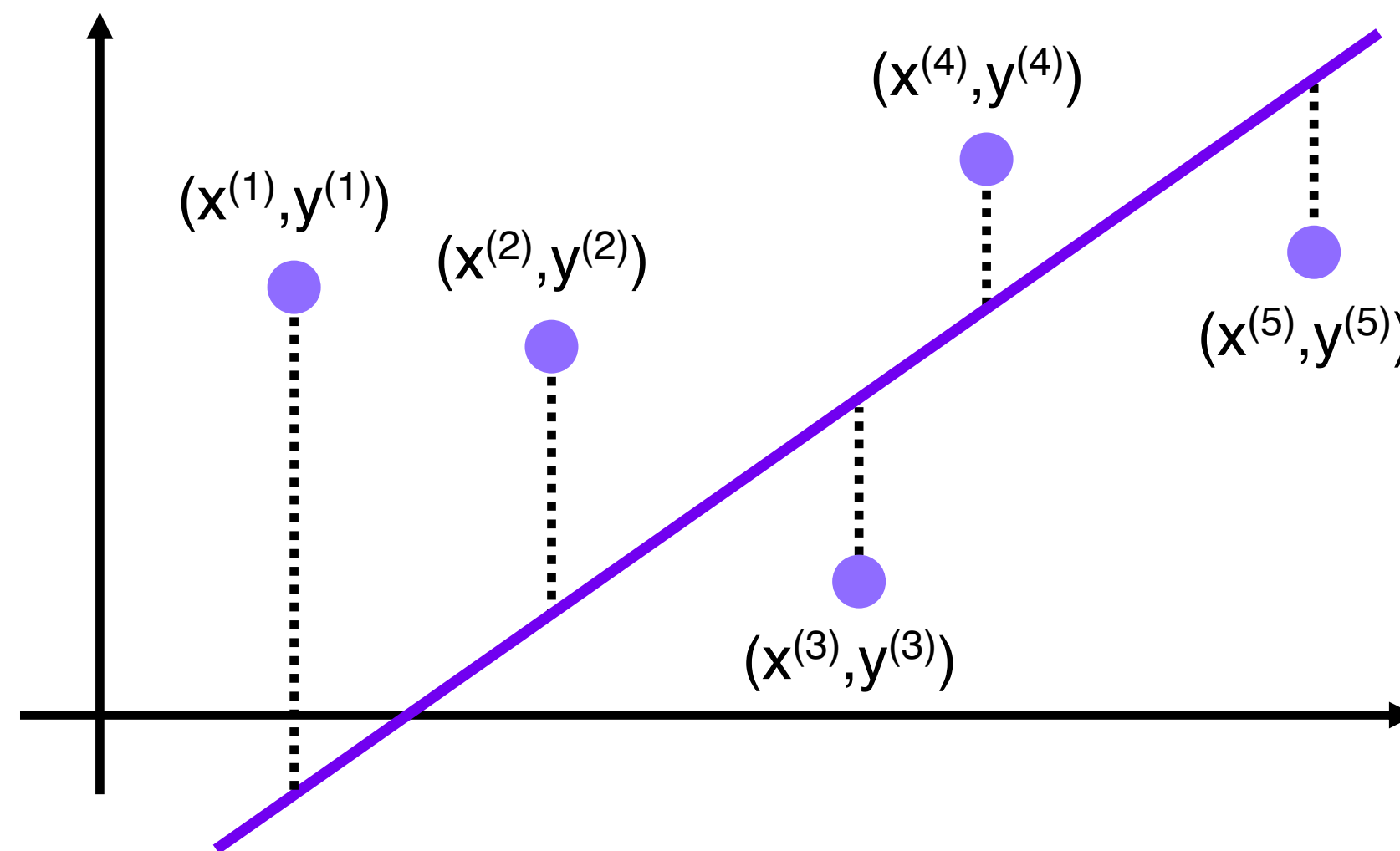


해당 구역에 있는 데이터들의 실제값(Y)과 예측값의 오차 제곱합을 최소화하는 구역

RSS(Residual Sum of Squares)

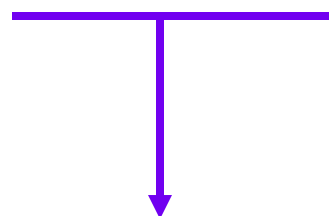


- Residual Sum of Squares
- 회귀 알고리즘에서 등장한 개념으로 **실제값과 예측값의 “단순 오차 제곱합”**





어떤 구역으로 어떻게 나누어야 하는가?



RSS를 최소화하는 모든 구역을 한 번에 찾는 것은 **계산적으로 불가능**

→ 위에서부터 순간순간마다 가장 최선의 선택을 하자

Top-down

Greedy Approach



Greedy Approach

→ 위에서부터 순간순간마다 가장 최선의 선택을 하자
Top-down Greedy Approach

- 가능한 모든 분할 기준을 시도해보고, 각 기준에 대해 분할 후의 결과를 평가
- 분할 후 하위 노드의 데이터들이 더 작고 더 균일한 값들로 묶이도록 하는 분할 기준 찾기



회귀 의사결정나무 구현

```
from sklearn.tree import DecisionTreeRegressor  
  
reg = DecisionTreeRegressor(max_depth = 5) 모델 생성  
  
reg.fit(X,y) 모델 학습  
  
reg.predict(X_test) 회귀 결과 반환
```

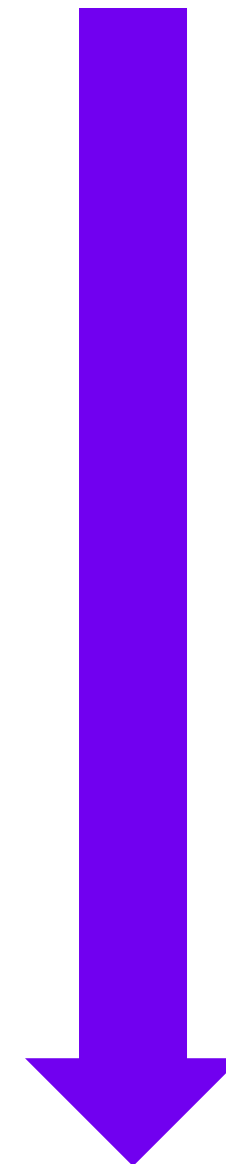
- max_depth = 트리의 최대 깊이
 - default = None



의사결정나무 - 분류

- 분류에서도 회귀 의사결정 나무를 그대로 적용할 수 있을까?

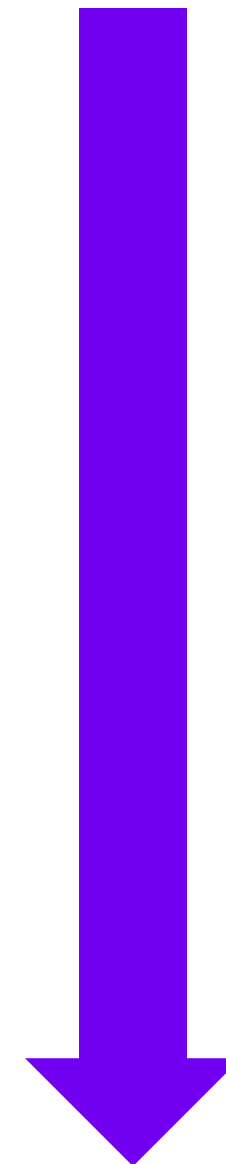
→ 분류 문제에서 **구하고자 하는 것은 해당 구역의 평균값이 아니기** 때문에 **RSS를 사용할 수 없음**





불순도(Impurity)

- 데이터의 불순도(Impurity)를 최소화하는 구역으로 나누자!

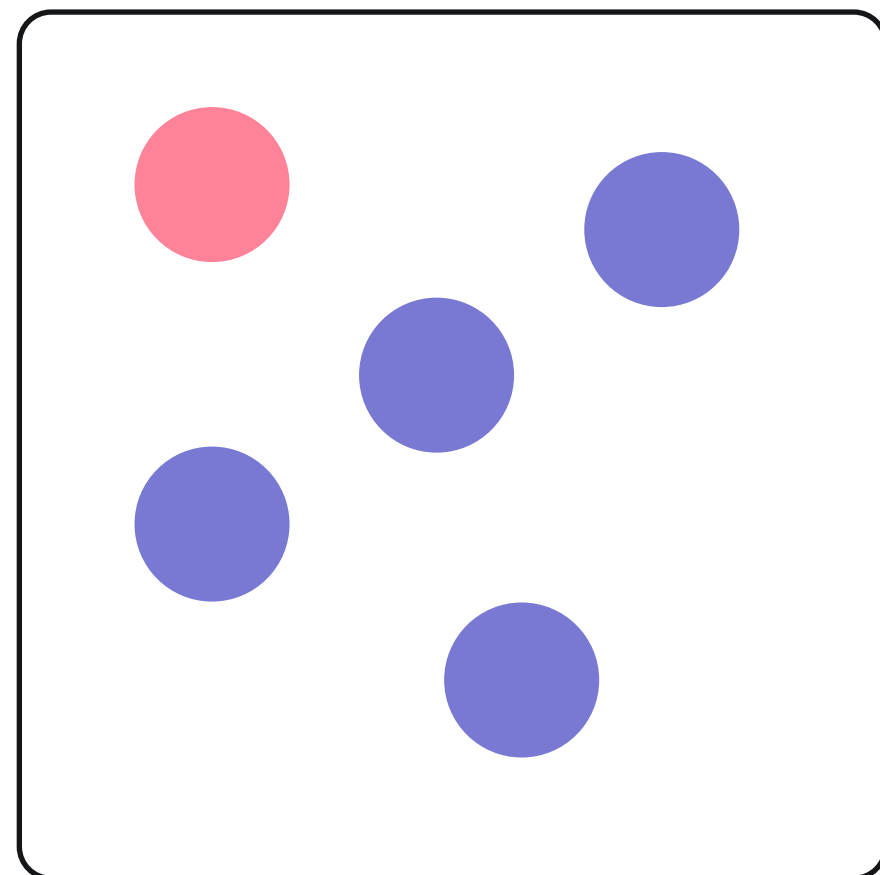




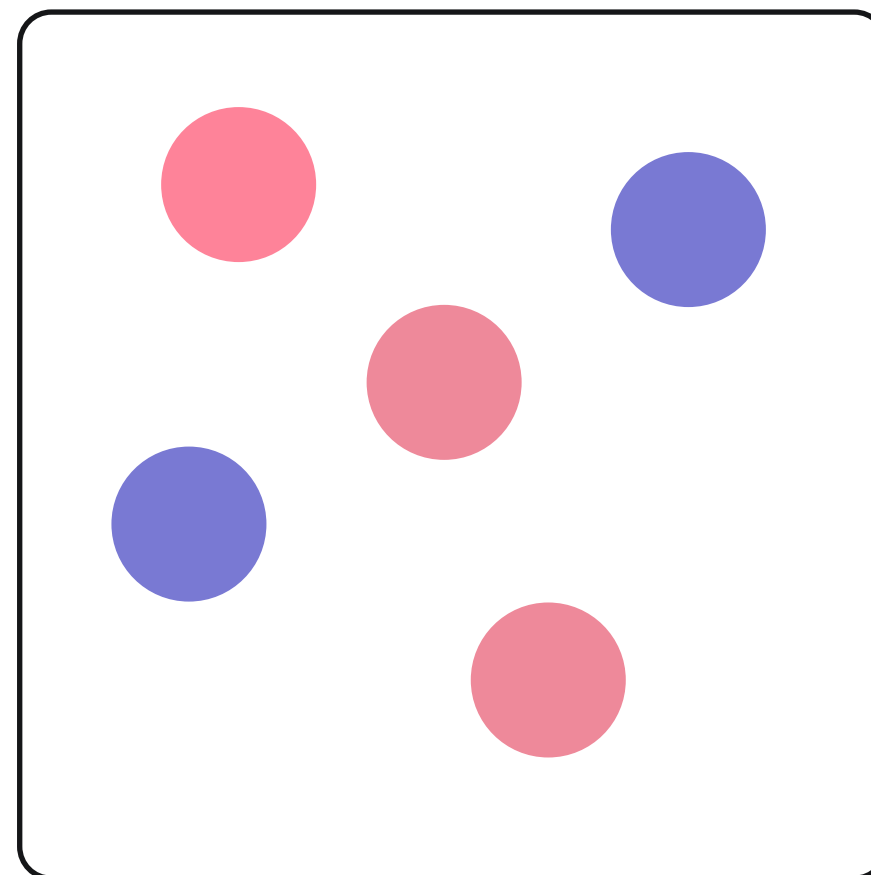
불순도(Impurity)

다른 데이터가 섞여 있는 정도

- 불순도가 낮을수록 노드가 순수해지며, 대부분의 데이터가 동일한 클래스로 이루어졌다는 뜻
- 지니 지수, 엔트로피 등을 사용



A



B

불순도

$A < B$



지니 지수 (Gini Index)

해당 구역 안에서 특정 클래스에 속하는 데이터의 비율을 모두 제외한 값

- 다양성을 계산하는 방법
- Yes or No의 이진 분류일 때,

$$Gini\ Index = 1 - (P(Yes))^2 - (P(No))^2$$



정보 이론에 기반한 척도로, 데이터가 얼마나 예측하기 어려운지를 나타냄

- 노드 내 클래스의 불확실성을 측정하는 방법
- $Entropy = -\sum_{i=1}^C p_i \log_2 p_i$



분류 의사결정나무 구현

```
from sklearn.tree import DecisionTreeClassifier
```

```
clf = DecisionTreeClassifier(criterion='gini') 모델 생성
```

```
clf.fit(X,y) 모델 학습
```

```
clf.predict(X_test) 분류 결과 반환
```

- criterion = 분류하는 기준
 - default = 'gini'
 - {'gini', 'entropy', 'log_loss'}

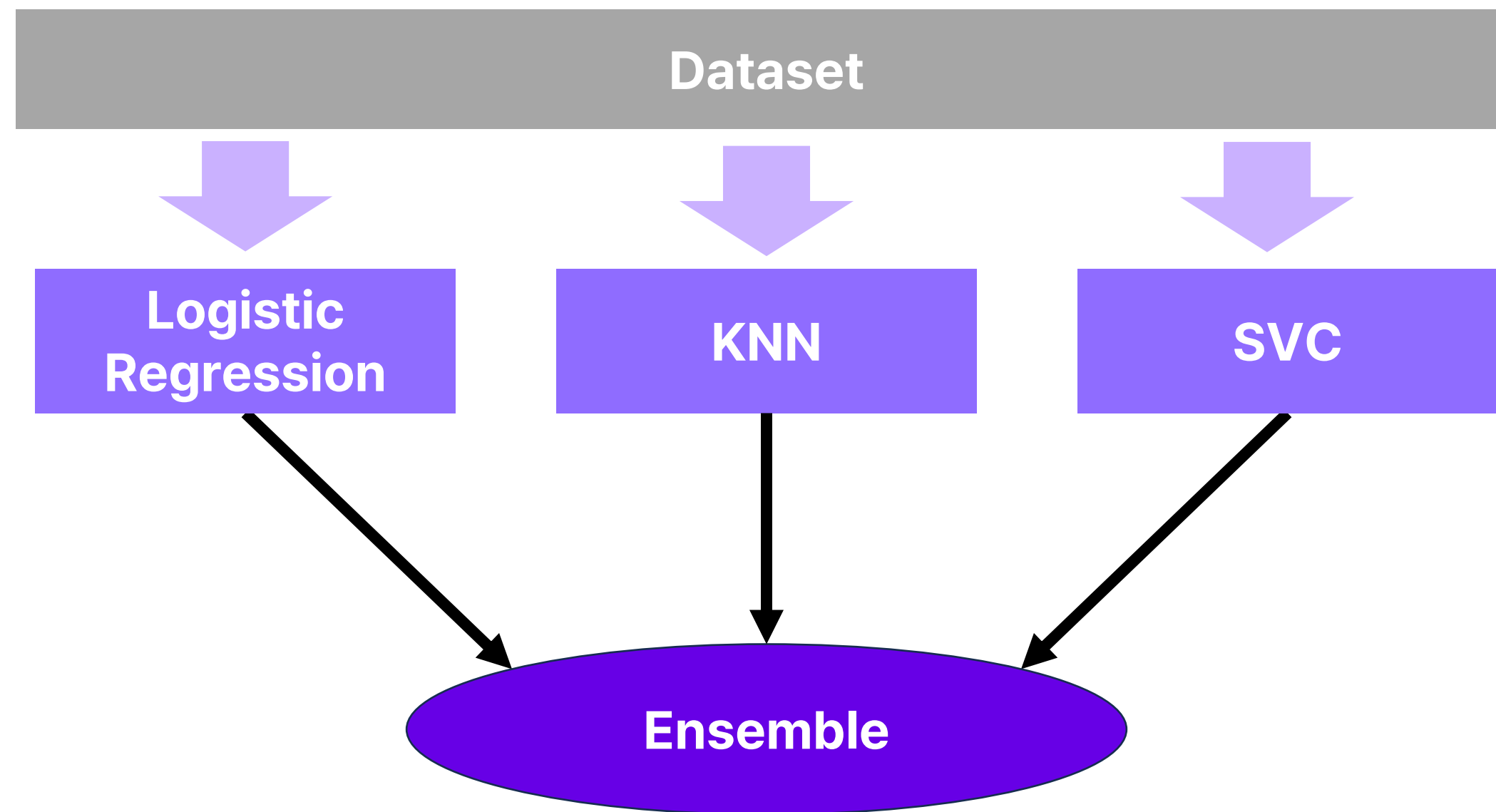


과적합 방지 및 더 높은 성능의 결과를 도출하도록 여러 모델을 활용하는 기법

- 데이터가 복잡하거나 노이즈가 많은 경우 유용
- 대표적인 앙상블 기법 : Voting, Bagging(Bootstrap Aggregating), Boosting

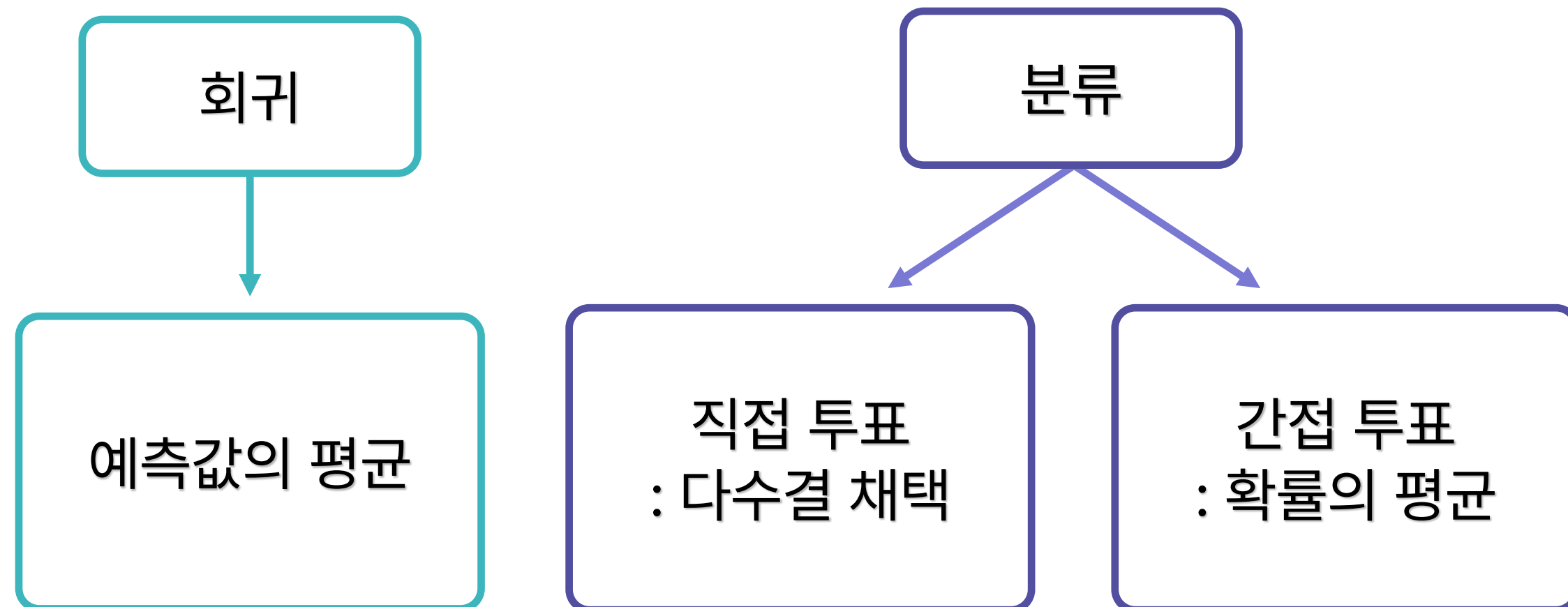


여러 모델의 예측 결과값을 활용하여 **투표를 통해** 최종 예측값을 결정





- 회귀의 경우
 - 각 모델의 예측값 평균
- 분류의 경우
 - 직접 투표 : 다수결 채택 (하드 보팅)
 - 간접 투표 : 확률의 평균 (소프트 보팅)





보팅의 특징

- 결과가 **직관적**이며 해석이 쉬움
- 평균, 다수결 채택 등의 **간단한 연산**을 사용
- **서로 다른 알고리즘**을 사용한 여러 모델을 결합할 수 있음



```
# VotingClassifier 에 사용할 모델들도 import
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import VotingClassifier
```

```
lr_clf = LogisticRegression()
knn_clf = KNeighborsClassifier()
```

모델 생성

```
vo_clf = VotingClassifier(estimators = [('LR', lr_clf), ('KNN', knn_clf)], voting='soft')
```

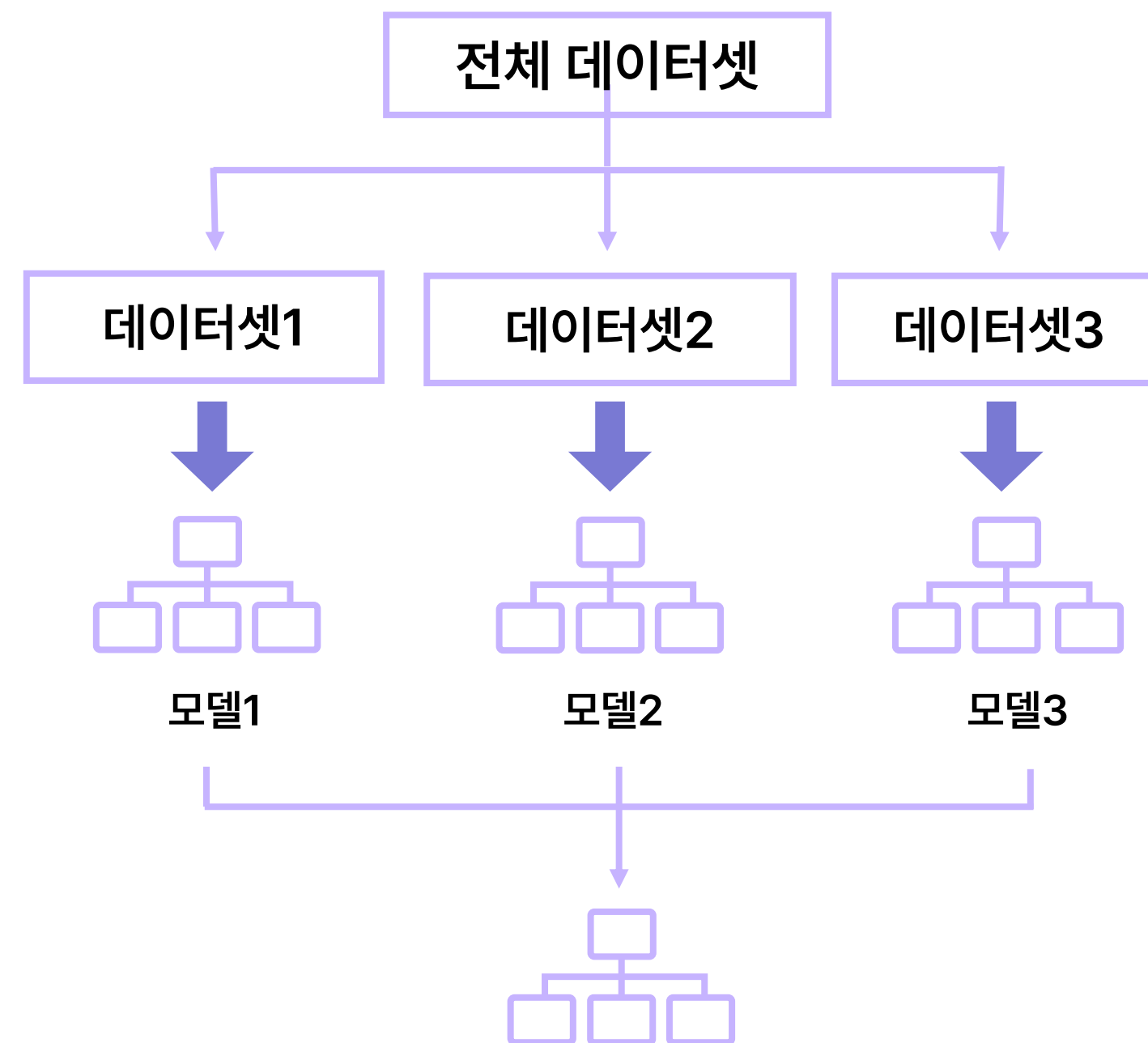
```
vo_clf.fit(train_X, train_y) 모델 학습
```

```
vo_clf.predict(test_X) 보팅 분류 결과 반환
```

- estimators = 앙상블 학습을 진행할 분류 모델 설정
 - [('모델 이름1', 모델1), ('모델 이름2', 모델2)...] 형식으로 전달
- voting = 보팅 방식 설정
 - default = 'hard'
 - {'hard', 'soft'}



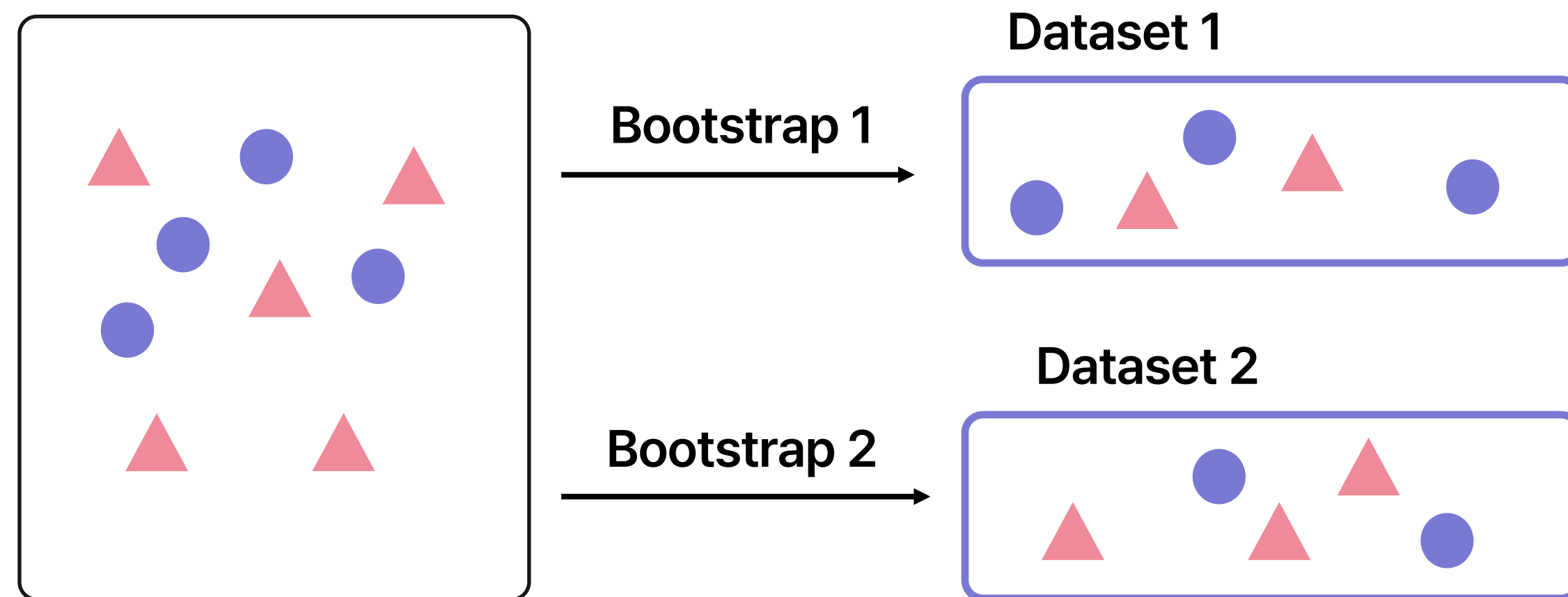
복원 추출을 통해 랜덤 추출한 데이터를 생성하고, 각 데이터를 모델 학습하여 결합한 후,
학습된 모델의 예측 변수를 활용하여 최종 모델을 생성하는 방법





Bootstrap

- Bagging = **Bootstrap** + Aggregating
- 데이터를 **복원 추출**하는 통계학적 방법
- 데이터를 추출한 이후 다시 원 데이터로 복원한 후 다시 데이터셋을 추출





배깅의 특징

- Bootstrap 과정을 활용한 **랜덤 샘플링으로 과적합 방지**
- 복원 과정이 있어 **불균형하거나 충분하지 않은 데이터에 적용** 가능
- 모델들이 **서로 다른 데이터를 학습하여 각 모델에 다양성**을 부여할 수 있음



```
from sklearn.ensemble import BaggingClassifier  
from sklearn.tree import DecisionTreeClassifier
```

모델 생성

```
ba_clf = BaggingClassifier(base_estimator = DecisionTreeClassifier(), n_estimators=100)
```

```
ba_clf.fit(train_X, train_y)
```

 모델 학습

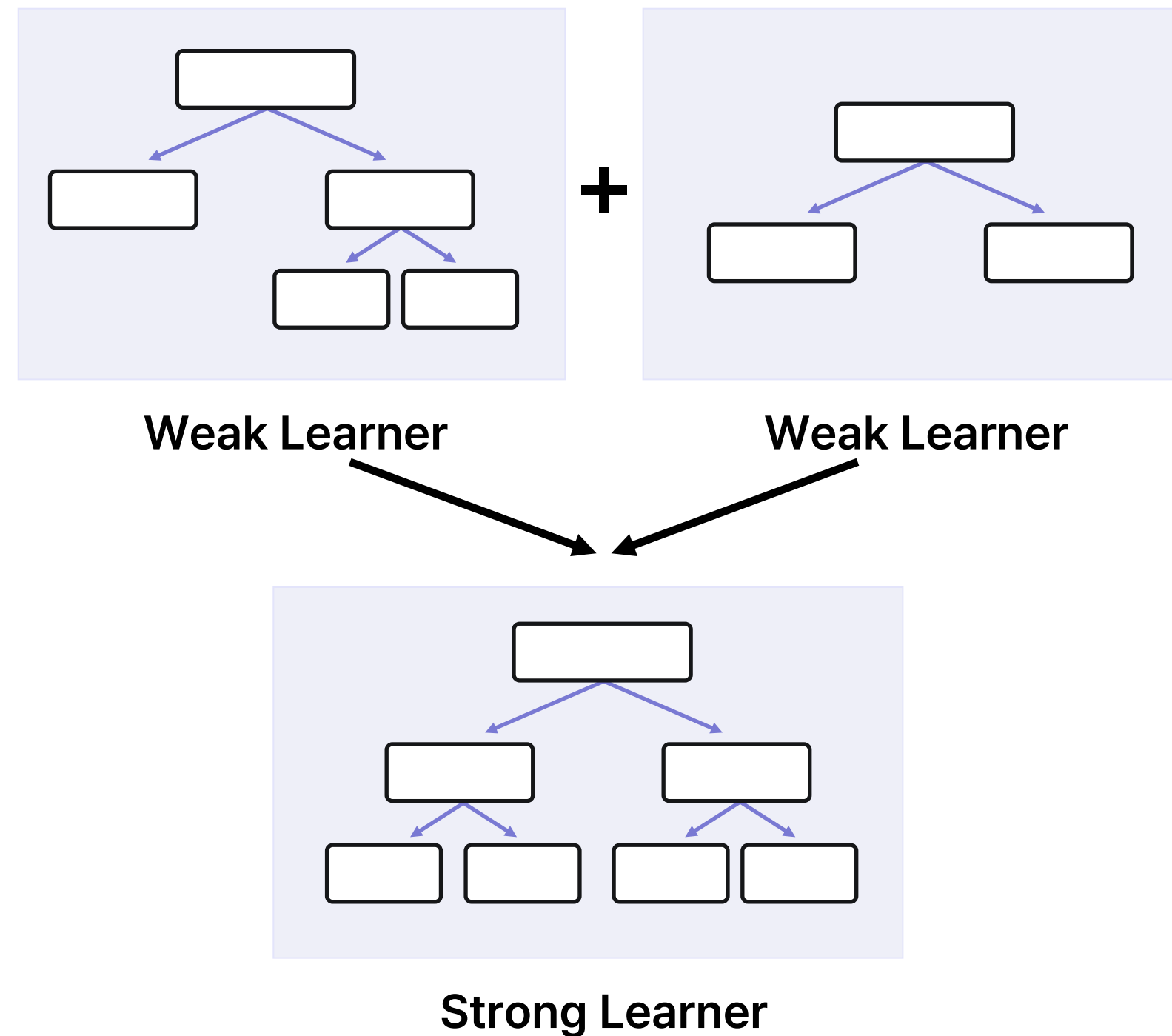
```
ba_clf.predict(test_X)
```

 배깅 분류 결과 반환

- base_estimator = 앙상블을 진행할 분류 모델 설정
- n_estimators = 사용하고자 하는 분류 모델의 개수
 - default = 10



여러 개의 **약한 모델(Weak Learner)**을 수정하여
강한 모델(Strong Learner)을 만드는 방법





- 약한 모델(Weak Learner) : 상대적으로 예측 성능이 떨어지는 모델
- 독립적인 모델을 합산하여 산출하기보다는 **기존의 모델을 개선**시키는 방향의 앙상블 기법
- 여러 개의 분류기가 순차적으로 학습을 수행하되,
앞서 학습한 분류기가 **잘못 예측한 데이터에 대해서는 다음 분류기에 가중치**를 부여



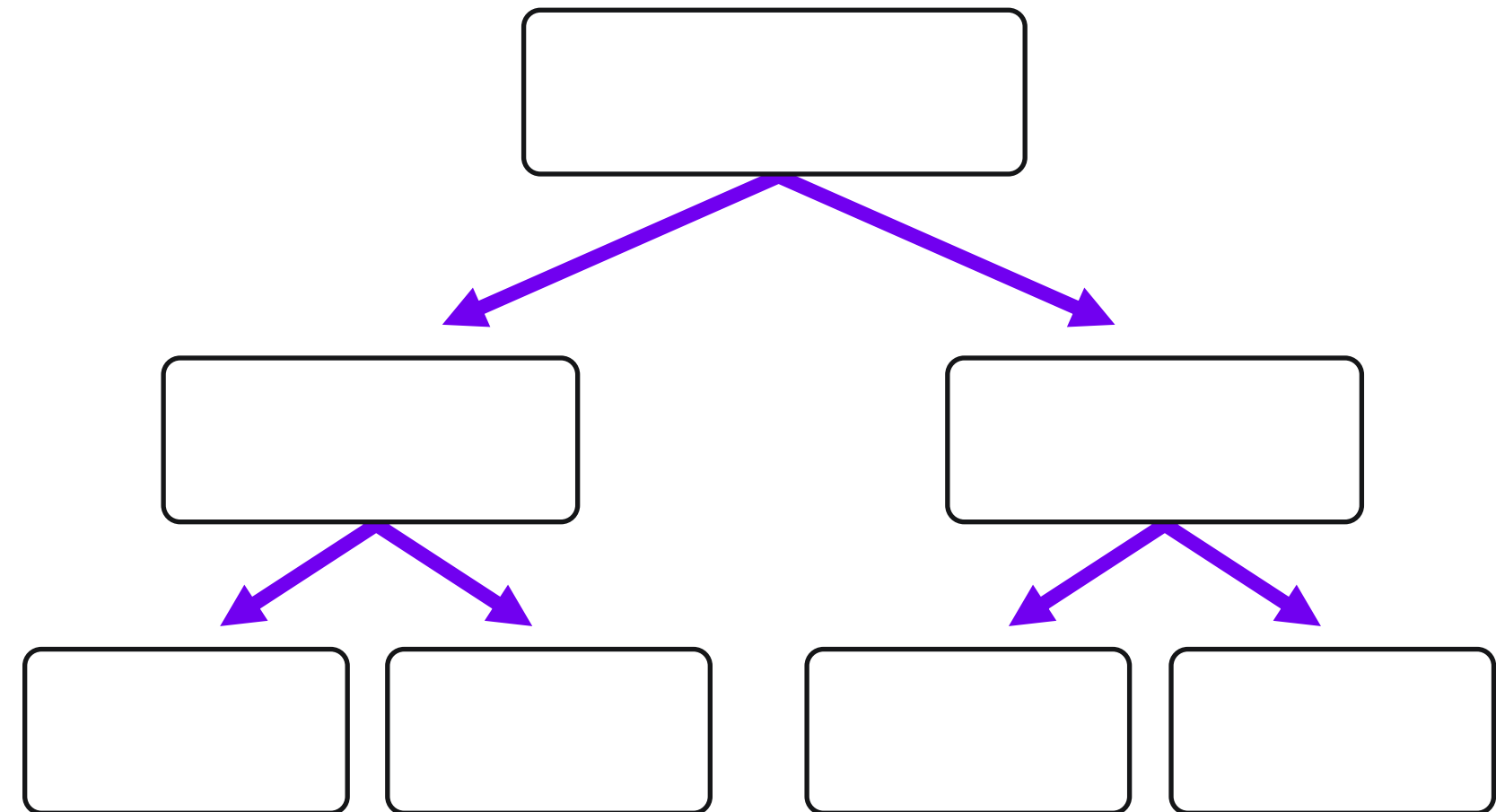
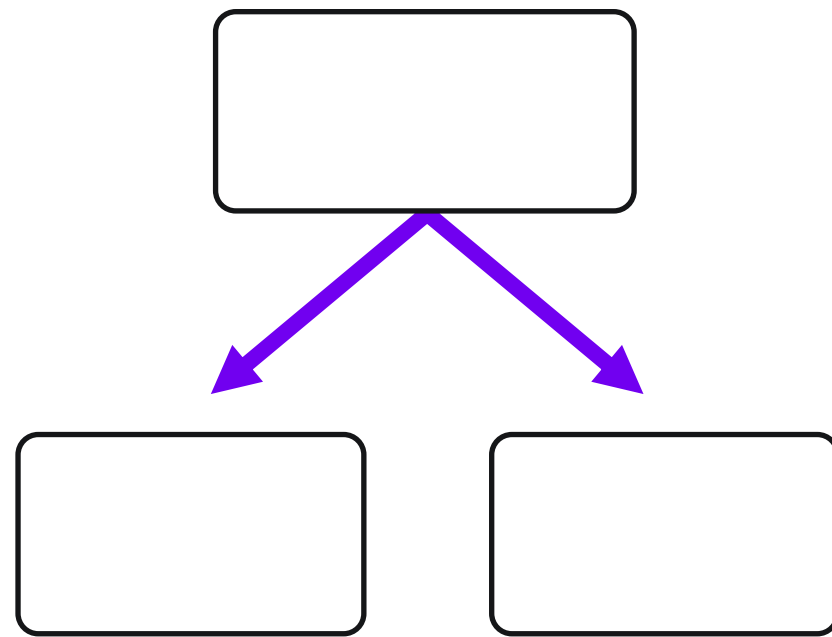
앙상블 기법의 다양한 모델

- 의사결정트리에 앙상블 기법을 활용한 다양한 모델들이 존재
- 대표적인 모델
 - 랜덤 포레스트, 부스팅 계열 모델
(Ada Boost, Gradient Boosting, XGBoost, LGBM, CatBoost)



의사결정나무의 한계

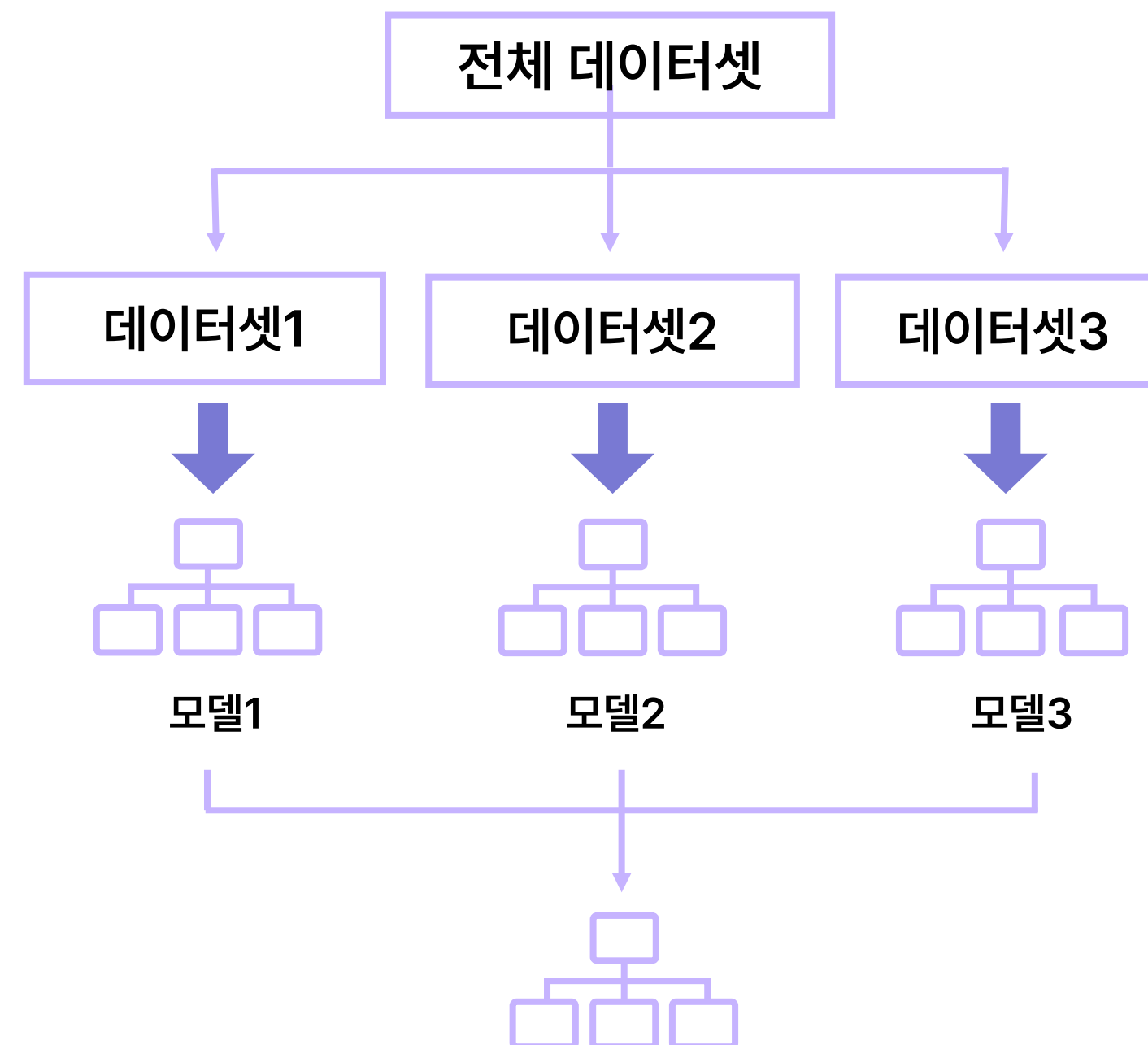
- 단일 모델 사용 시, 과적합이 나타날 가능성이 높다면 **여러 개의 의사결정나무**를 사용해보자
= 랜덤 포레스트





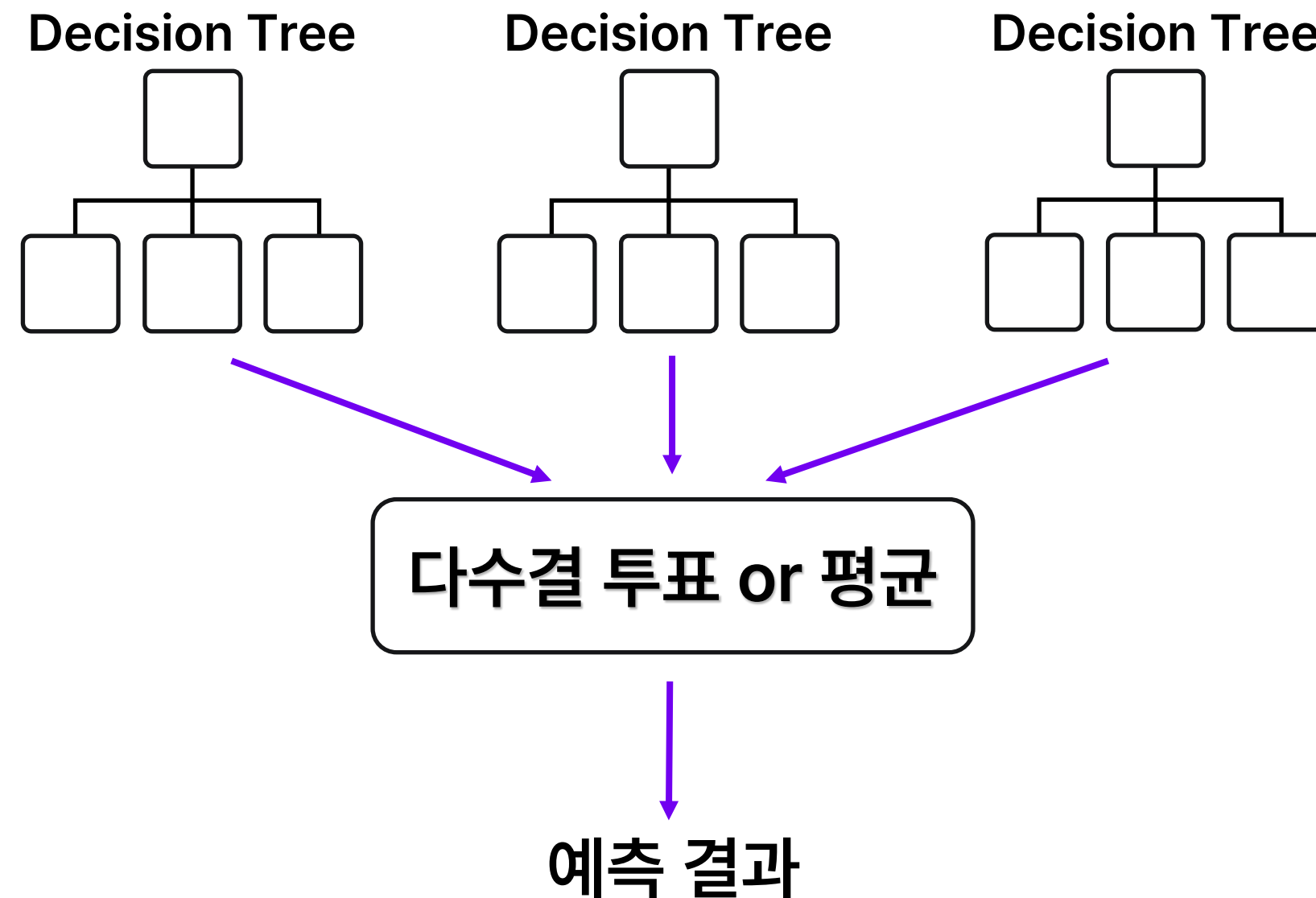
의사결정트리 + Bagging 알고리즘

여러 개의 나무를 사용해 숲을 생성한다는 의미





- 데이터셋에서 **Bootstrap을 통해 N개의 훈련 데이터셋**을 생성
 - 개별 데이터에 대해서만 복원 추출이 아닌 입력 변수에 대해서 복원 추출 함께 진행
- 생성한 N개의 의사결정나무를 학습함
- 학습된 나무들의 예측 결과값의 평균 또는 다수결 투표 방식 이용하여 결합





랜덤포레스트 특징

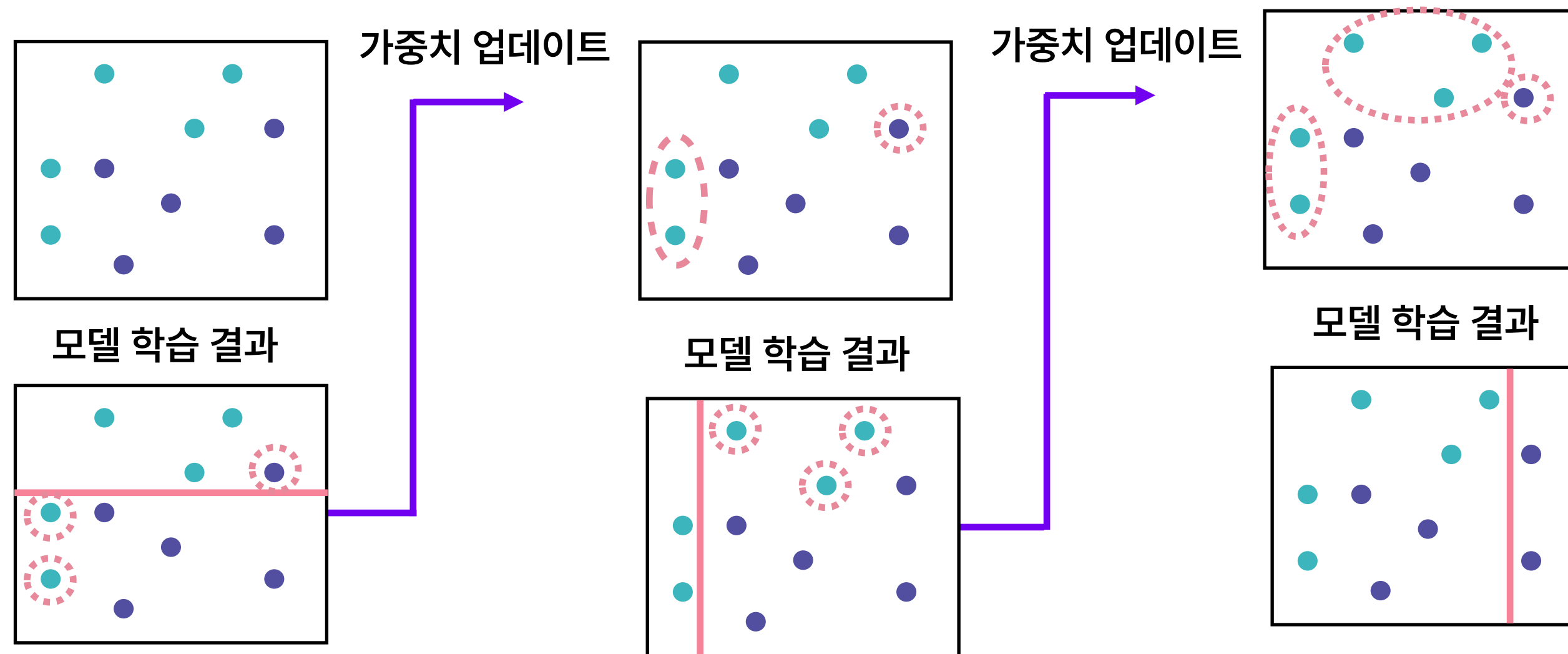
- 의사결정나무와 동일하게 회귀와 분류 모두 적용 가능
- 각 트리에서 사용된 특성의 중요도를 추정하여 변수 선택 및 해석 유용
- 변수 일부를 사용하기 때문에 과적합을 방지할 수 있음
- 안정적이고 강력한 모델로, 일반적으로 높은 성능을 보임



Adaptive Boosting (적응 부스팅)

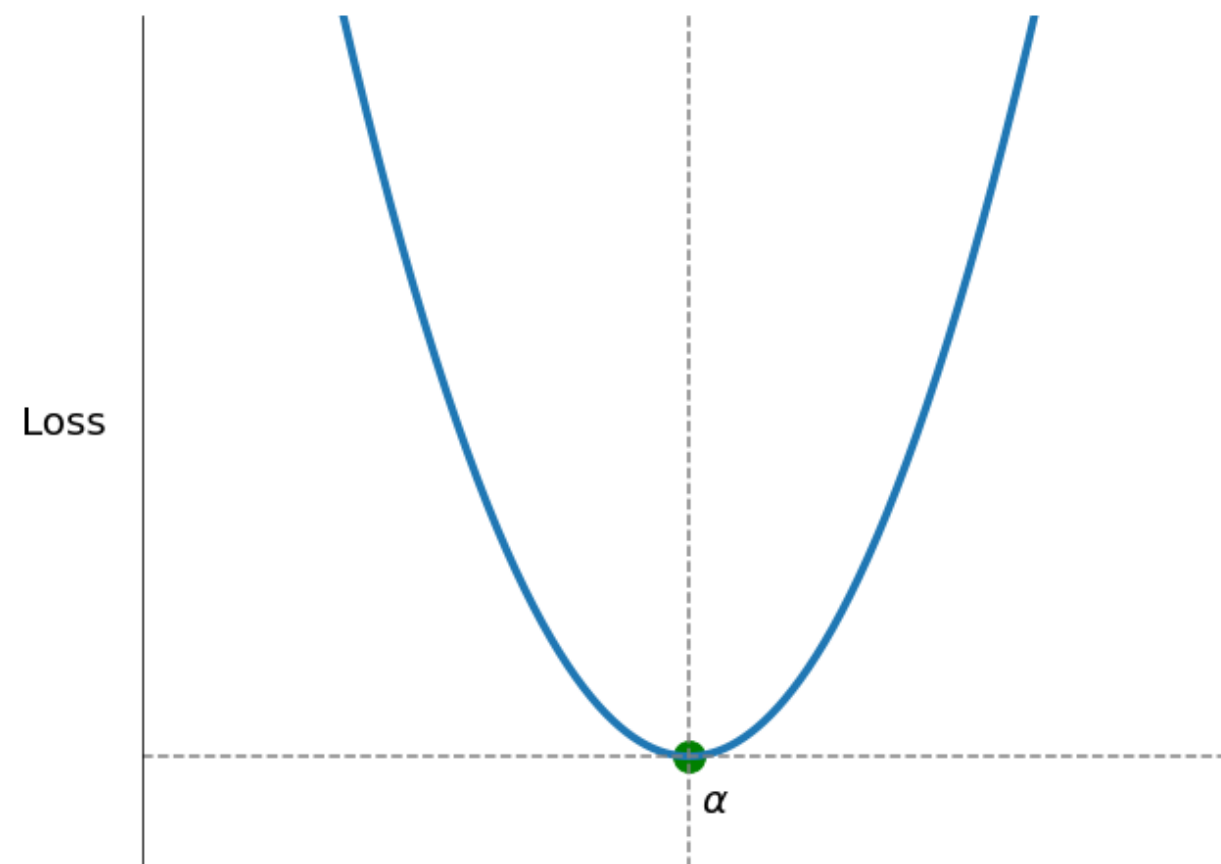
이전 학습 과정에서 오분류한 데이터를

다음 학습 과정에서는 잘 분류할 수 있도록 반영해 나가는 알고리즘





오차는 손실함수로 표현되고, 이 손실함수를 최적화하는 과정



오차값을 최소화하는 모델의 인자를 찾는 것



Ada boost 특징

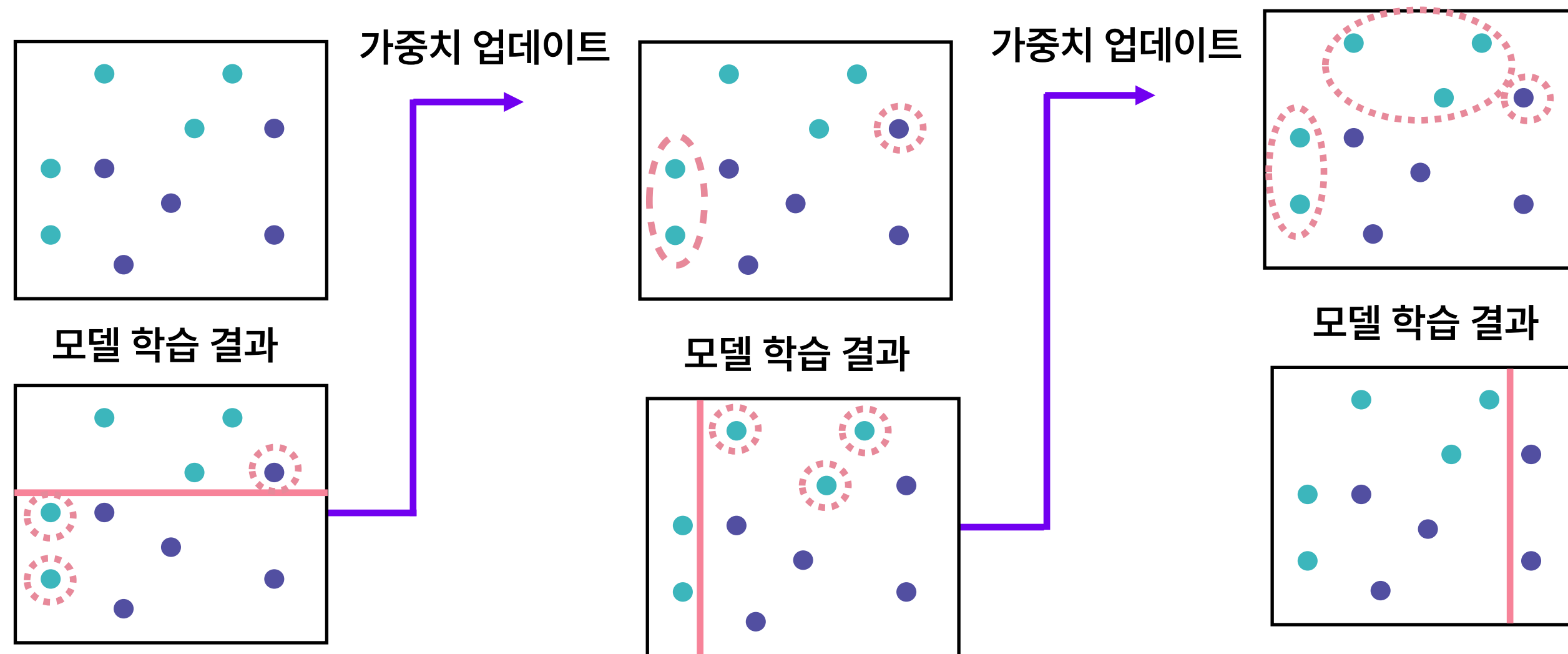
- 오분류 데이터에 대해 모델을 적합할 수 있음
- 계산 과정에 있어 병렬 수행이 불가능함
- 다양한 약한 모델(Weak Learner)과 유연하게 결합하며 강한 모델을 생성
- 가중치 업데이트 메커니즘을 통해 일반화 성능을 높여 잘 과적합하지 않음



Gradient Descent + Boosting

Ada Boost 와 동일한 원리로,

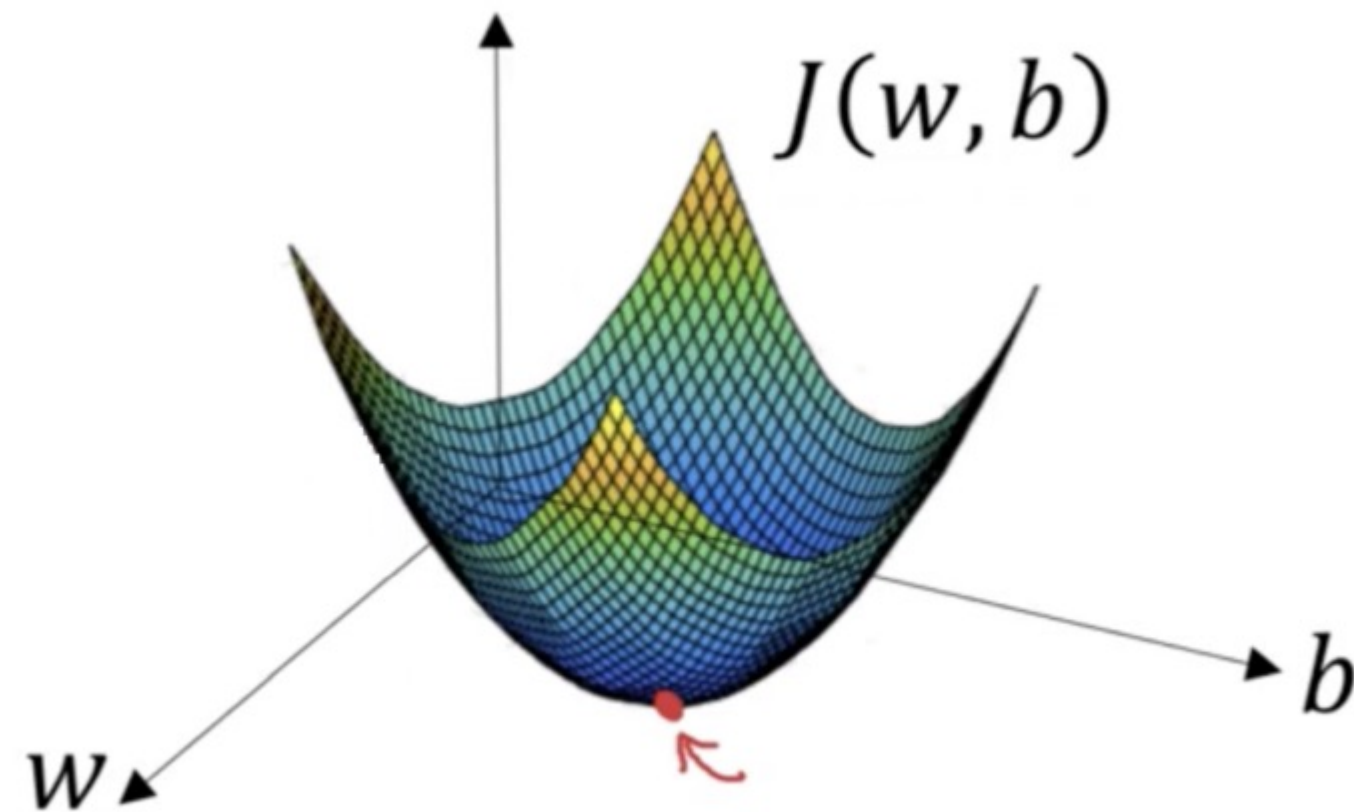
가중치 업데이트 과정에서 **Gradient Descent** 를 사용





Gradient Descent

함수의 기울기(Gradient)를 이용하여, 최소값을 찾는 최적화 알고리즘





Gradient Boosting 특징

- 계산량이 많이 필요하나 높은 성능을 도출
- 효율적 연산이 가능하도록 파이썬 라이브러리 내에 정의됨
- **Gradient Boosting을 응용한 다양한 모델**들이 개발됨
 - eXtreme Gradient Boosting(XGBoost)
 - Light Gradient Boosting Model(LGBM)
 - Categorical Boosting(CatBoost)



Gradient Boosting 기반 모델 소개

- Gradient Boosting 기반의 성능이 향상된 다양한 모델이 개발되어 간단한 모듈 불러오기만으로 사용이 가능

```
import xgboost as xgb      # XGBoost
import lightgbm as lgb     # LightGBM
from catboost import CatBoostClassifier, CatBoostRegressor      # CatBoost
```




eXtreme Gradient Boosting (XGBoost)

- 일반 Gradient Boosting 모델과 작동 원리는 동일
- 과적합을 방지하기 위해 **정규화된 모델을 사용**
 - 불순도를 감소하는 방향만 강조되는 트리에서 정규화를 추가하여 모델의 복잡성 고려





eXtreme Gradient Boosting (XGBoost) 특징

- 분산/병렬 처리를 통해 실행 속도를 보완
- 대부분의 문제에서 양호한 예측 성능
- 다양한 하이퍼파라미터 지원 및 조절을 통해 과적합 방지에 효율적



Light Gradient Boosting Model(LGBM)

- XGBoost에 비해 더 가볍고 빠른 실행 속도를 가진 모델
- 범주형 변수 처리 지원 기능 추가





Light Gradient Boosting Model(LGBM) 특징

- 대용량 데이터에서의 **실행 속도 개선**
- 기존 XGBoost 대비 **적은 메모리 사용**
- 적은 수의 데이터 적용 시, **과적합 문제 발생 가능성**



Categorical Boosting(CatBoost)

- Categorical + Boosting Model
- 범주형 변수를 위한 다양한 기능을 지원하는 부스팅 모델



Categorical Boosting (CatBoost) 특징

- 범주형 변수에 대한 전처리 문제 해결
- 범주형 변수를 자동으로 처리해 타입 변환 오류를 피할 수 있음
- 범주형 변수와 수치형 변수들의 Combination을 처리
- 변수 간의 상관관계를 계산함과 동시에 속도 개선
- Multiple-category 데이터를 다룰 때 유용

